

Signale und kontrollierte Zustellung

Vom Ereignis bis zum Handler und zurück in den unterbrochenen Kontext

5.08

Drei Fragen führen durch den Signalpfad

01 Wo entsteht ein Signal?

User Space, synchrone Fehler, Timer und Kernelereignisse besitzen unterschiedliche Quellen.

02 Wann wird es zugestellt?

Pending-Zustand, Ziel-Thread, Maske und Disposition bestimmen den Übergang zum Handler.

03 Was geschieht mit einem blockierten Syscall?

Der Kernel beendet, startet neu oder meldet Teilfortschritt – abhängig vom konkreten Vertrag.

Lernziel

Am Ende kannst du Entstehung, Zustellung und Rückkehr eines Signals aus Kernelsicht erklären.

Vier Phasen machen Signalzustellung kontrollierbar

Erzeugen

Quelle, Signalnummer, Ziel und optionale Zusatzdaten werden festgelegt.

Pending

Das Signal bleibt vorgemerkt, solange es maskiert oder noch nicht zustellbar ist.

Zustellen

Ein geeigneter Thread erhält Default-Aktion, Ignore oder einen User-Space-Handler.

Fortsetzen

Der Kernel stellt Register, Maske und unterbrochenen Ausführungskontext wieder her.

SIGNALPFAD

Erzeugen → Pending → Auswählen → Signalframe → Handler → Rückkehr

Entscheider: Ziel · Maske · Disposition · aktueller Kontext

Kernelstrukturen trennen Ziel, Disposition und Pending-Zustand

DATENMODELL

```
task_struct  
signal_struct  
sighand_struct  
sigpending  
sigset_t  
siginfo_t
```

Drei Verantwortungsbereiche

01

Ziel

task_struct trägt threadlokalen Signalzustand und die individuelle Maske.

02

Disposition

sighand_struct enthält gemeinsame Aktionen: Default, Ignore oder Handler.

03

Pending

Private und gemeinsame Queues halten Signale samt siginfo.

Signale entstehen durch Software, Fehler oder Zeitablauf

SIGNALQUELLEN

```
kill() / tgkill()  
sigqueue()  
Terminal + Job Control  
Timerablauf  
Hardwareausnahme  
Kernelereignis
```

Drei Klassen

01 Synchron

SIGSEGV, SIGILL oder SIGFPE beziehen sich direkt auf die laufende Instruktion.

02 Asynchron

Andere Tasks, Terminal, Timer oder Kernelpfade lösen ein späteres Ereignis aus.

03 Autorisierung

User Space darf nur zulässige Ziele und Signalnummern ansprechen.

Prozess- und threadgerichtete Signale wählen unterschiedlich

ZIELWAHL

process-directed

gemeinsames Pending

geeigneter Thread

thread-directed

privates Pending

fester Thread

Maske: pro Thread

Disposition: gemeinsam

Zwei Ebenen

Prozessgerichtet

Der Kernel wählt einen Thread der Gruppe, der das Signal nicht blockiert.

Threadgerichtet

tgkill(), pthread_kill() und synchrone Fehler binden das Signal an einen Thread.

Ein Prozess besitzt gemeinsame Signalaktionen – jeder Thread kontrolliert seine eigene Maske.

Zustellung verändert kontrolliert den Rückweg in den User Space

RÜCKKEHRPFAD

```
Kernel exit path
    pending & ~blocked
    dequeue_signal()
    setup_rt_frame()
User handler(signo,
    siginfo, ucontext)
    rt_sigreturn()
```

Lies den Übergang

01

Auswahl

Nur nicht blockierte Signale sind für diesen Thread zustellbar.

02

Signalframe

Register, Maske und Rückkehrdaten landen kontrolliert auf dem User-Stack.

03

Fortsetzung

`rt_sigreturn()` stellt den gesicherten Kontext wieder her.

Der Handler läuft im User Space; der Kernel konstruiert und validiert den Übergang.

Standard- und Echtzeitsignale haben verschiedene Queue-Semantik

ZWEI KLASSEN

Standard

- Instanzen können zusammenfallen
- kein Nutzdatenstrom

Real-time

- mehrere Instanzen
- `sigqueue()`-Payload
- definierte Priorität
- `SIGRTMIN + n`

Drei Konsequenzen

01 Queue

Standard-Signale merken typischerweise nur „mindestens einmal pending“.

02 Daten

Echtzeitsignale halten mehrere `siginfo`-Einträge samt Wert.

03 Reihenfolge

Gleiche Echtzeitsignale werden in Erzeugungsreihenfolge zugestellt.

Disposition und Maske bilden den Zustellungsvertrag

SIGACTION

```
struct sigaction sa = {  
    .sa_sigaction = handler,  
    .sa_flags = SA_SIGINFO |  
                SA_RESTART,  
};  
sigemptyset(&sa.sa_mask);  
sigaction(SIGTERM, &sa, NULL);
```

Drei Entscheidungen

01

Aktion

Default, Ignore oder Handler bestimmen die Wirkung.

02

Maske

Thread- und Handler-Maske verschieben die Zustellung.

03

Restart

SA_RESTART startet ausgewählte Syscalls transparent neu.

Unterbrechbare Wartepfade reagieren auf Ereignis oder Signal

KERNEL-CODE

```
ret = wait_event_interruptible(wq,  
    ready || gone);  
  
if (ret)  
    return ret;    /* -ERESTARTSYS */  
  
if (gone)  
    return -ENODEV;  
  
return consume_data();
```

Zwei Ausgänge

Ereignis

Die Bedingung wird wahr; der Treiber prüft den Zustand erneut und arbeitet weiter.

Signal

Die Task wird geweckt; der Pfad räumt auf und gibt den Restart-Code zurück.

Jeder unterbrochene Pfad muss Locks, Referenzen und Teilergebnisse korrekt behandeln.

Syscall-Restart ist eine Policy zwischen Kernel und Handler

RESTARTPFAD

Kernel intern

- ERESTARTSYS
- ERESTARTNOHAND
- ERESTART_RESTARTBLOCK

User Space

- automatischer Neustart
- oder -1 / EINTR
- oder Teilfortschritt

Drei Fälle

01

Handler

Der Rückkehrpfad entscheidet, ob der Syscall neu angesetzt wird.

02

EINTR

Ohne Restart gibt der Aufruf Kontrolle sichtbar an die Anwendung zurück.

03

Teilfortschritt

Bereits übertragene Bytes dürfen nicht dupliziert werden.

Anwendungen müssen EINTR und partielle I/O-Ergebnisse als normalen Kontrollfluss behandeln.

Die Ereignisschnittstelle muss zum Datenfluss passen

SCHNITTSTELLEN

`poll / epoll`

Bereitschaft

`eventfd`

Ereigniszähler

`signalfd`

Signale als Records

`fasync / SIGIO`

asynchrone Benachrichtigung

Drei Auswahlregeln

01 Readiness

`poll` und `epoll` melden Bereitschaft; Daten folgen über `read()` oder `mmap()`.

02 Synchron integrieren

`signalfd` macht maskierte Signale im fd-basierten Eventloop lesbar.

03 Gezielt einsetzen

Signale steuern Kontrollfluss; sie ersetzen keinen Datenkanal.

Praxis: Zustellung an einer eigenen Test-Task beobachten

TERMINAL

```
$ sleep 60 & pid=$!  
$ grep -E 'State|Sig(Pnd|Blk|Ign|Cgt)' \  
    /proc/$pid/status  
$ kill -STOP "$pid"  
$ ps -o pid,state,cmd -p "$pid"  
$ kill -CONT "$pid"  
$ kill -TERM "$pid"  
$ wait "$pid"
```

Belege drei Aussagen

- 01 Disposition**
/proc zeigt Maske, Ignore und Handler als Bitfelder.
- 02 Zustellung**
SIGSTOP setzt Zustand T; SIGCONT lässt die Task weiterlaufen.
- 03 Cleanup**
SIGTERM beendet; wait() räumt den Shell-Job auf.

Signale verbinden Ereignis, Task-Zustand und Rückkehr.

Formuliere in einem Satz:

Welche vier Entscheidungen bestimmen, ob und wie ein pending Signal wirksam wird?